

:

WEATHER FORECASTING USING MACHINE LEARNING

A Project Report

Submitted by:

Name : Meet Ravindra Koli

Registration Number: 26BME10002

Course:

CSE1021 Introduction to Problem Solving and Programming

Semester:

1st Semester

College:

VIT Bhopal University

Academic Year:

2026-27

1. Project Overview

Project Title

Weather Forecasting Using Machine Learning with Python

Project Objective

The objective of this project is to develop a simple machine-learning-based weather forecasting system that uses historical weather observations to predict the next day's temperature.

The project demonstrates:

- Python programming
- Data collection and preprocessing

- Exploratory data analysis
- Data visualization
- Machine learning
- Model training
- Model evaluation
- Prediction
- Basic software architecture
- Testing
- Git/GitHub usage

The system is designed specifically to be understandable and implementable by a 1st-semester student.

2. Problem Statement

Weather conditions change continuously and are influenced by several factors such as temperature, humidity, pressure, wind speed, and rainfall.

Traditional weather forecasting systems can require large amounts of data and complex scientific models. This project explores a simpler approach using machine learning.

The proposed system takes historical weather data as input, processes the data, trains a machine-learning model, and predicts the temperature for the following day.

3. Scope of the Project

The project will:

- Read historical weather data from a CSV file.
- Clean and preprocess the data.
- Analyze weather information.
- Visualize important weather parameters.
- Train a machine-learning regression model.
- Evaluate the trained model.
- Accept weather information from the user.
- Predict the next day's temperature.
- Display the prediction.
- Store project-related data in files rather than requiring a complex database.

Outside the scope

The beginner version will not attempt to:

- Predict severe weather events.
- Replace professional meteorological systems.

- Provide guaranteed real-time forecasts.
- Predict every weather parameter simultaneously.
- Use deep-learning models.

4. Target Users

The system is primarily intended for:

- Students learning Python and Machine Learning
- Beginners interested in weather prediction
- Academic project demonstrations
- Teachers evaluating basic ML concepts

5. Functional Requirements

Your project has five major functional modules, which comfortably satisfies the requirement of at least three.

Module 1 — Data Input

The system should:

- Read weather data from a CSV file.
- Check whether required columns exist.
- Handle missing values.
- Convert data into a usable format.

Input: `weather.csv`

Output: Cleaned weather dataset.

Module 2 — Data Processing and Analysis

The system should:

- Remove or handle missing values.
- Convert dates into appropriate formats.
- Select relevant features.
- Display basic statistics.

Example features:

Temperature
Humidity
Pressure
WindSpeed
Rainfall

Module 3 — Data Visualization

The system should generate graphs such as:

- Temperature over time
- Humidity over time
- Temperature vs humidity

These graphs help the user understand the dataset before training the model.

Module 4 — Machine Learning Prediction

The system trains a regression model using historical weather information.

For a beginner project, use:

Linear Regression

The model learns the relationship between weather features and temperature.

Example:

```
Humidity = 65%  
Pressure = 1012 hPa  
Wind Speed = 8 km/h  
Rainfall = 2 mm
```

↓

```
Machine Learning Model
```

↓

```
Predicted Temperature = 27.4°C
```

Module 5 — Evaluation and Reporting

The system evaluates the model using:

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)

- R^2 Score

It can also display:

Actual Temperature
Predicted Temperature
Prediction Error

6. Input/Output Structure

Input

The primary input is a CSV file.

Example:

```
Date, Temperature, Humidity, Pressure, WindSpeed, Rainfall
2025-01-01, 23.5, 65, 1012, 7.2, 0
2025-01-02, 24.1, 63, 1010, 8.1, 0
2025-01-03, 25.3, 61, 1009, 6.8, 1.2
2025-01-04, 24.8, 67, 1011, 5.4, 0
```

Output

The program produces:

- Cleaned data
- Statistical information
- Graphs
- Model evaluation metrics
- Predicted temperature

Example:

Weather Forecasting System

```
Enter Humidity: 60
Enter Pressure: 1012
Enter Wind Speed: 7
Enter Rainfall: 0
```

```
Predicted Temperature: 26.35 °C
```

7. Non-Functional Requirements

You need at least four. You can include these six.

7.1 Performance

The system should process a small or medium-sized CSV dataset and generate predictions within a few seconds on a normal computer.

7.2 Usability

The application should have a simple command-line interface with clear instructions.

7.3 Reliability

The program should handle common problems such as:

- Missing values
- Invalid numerical input
- Missing CSV files
- Incorrect column names

7.4 Maintainability

The program should be divided into separate Python files instead of putting all functionality into one large file.

7.5 Resource Efficiency

The project should run on a normal student laptop without requiring a GPU.

7.6 Error Handling

The program should display understandable error messages rather than terminating unexpectedly when users enter invalid values.

8. Technology Stack

Technology	Purpose
Python	Main programming language
Pandas	Data processing

NumPy	Numerical operations
Matplotlib	Graphs
Scikit-learn	Machine learning
CSV	Data storage
Git	Version control
GitHub	Repository hosting

9. Machine Learning Model

Model Selection

For a first-semester project, I recommend Linear Regression.

Linear Regression is suitable because it is:

- Easy to understand
- Easy to implement
- Fast
- Appropriate for numerical prediction
- Easy to explain during viva

The basic idea is:

$$Y=b_0+b_1X_1+b_2X_2+...+b_nX_n$$

Where:

- Y = predicted temperature
- X = weather features
- b = model parameters

10. Dataset

You can use a historical weather dataset containing daily observations.

For the beginner version, keep these columns:

Date
Temperature
Humidity
Pressure
WindSpeed
Rainfall

Feature

The input features are:

Humidity
Pressure
WindSpeed
Rainfall

Target

Temperature

The model learns to predict:

Temperature

11. Model Evaluation

Split the dataset into training and testing portions.

For example:

80% → Training
20% → Testing

The training data teaches the model.

The testing data checks how well the model performs on unseen data.

Metrics

Mean Absolute Error

$$MAE = \frac{1}{n} \sum | \text{Actual} - \text{Predicted} |$$

It represents the average absolute prediction error.

Mean Squared Error

$$MSE = \frac{1}{n} \sum (Actual - Predicted)^2$$

Root Mean Squared Error

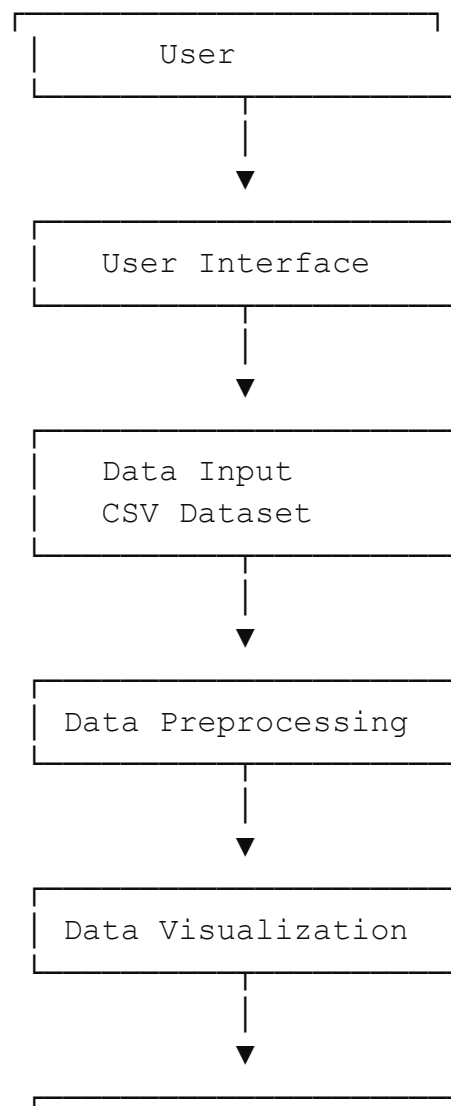
$$RMSE = \sqrt{MSE}$$

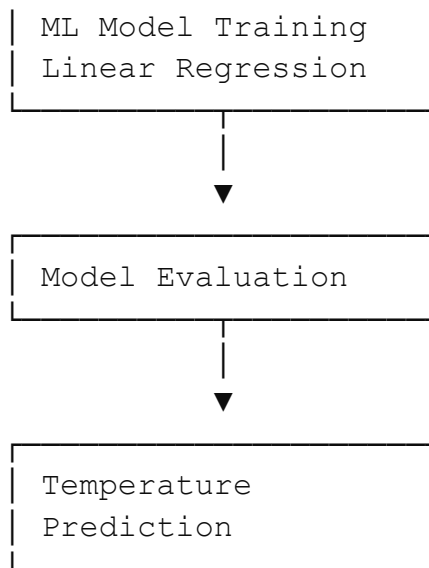
R² Score

R² indicates how much of the variation in the target variable is explained by the model.

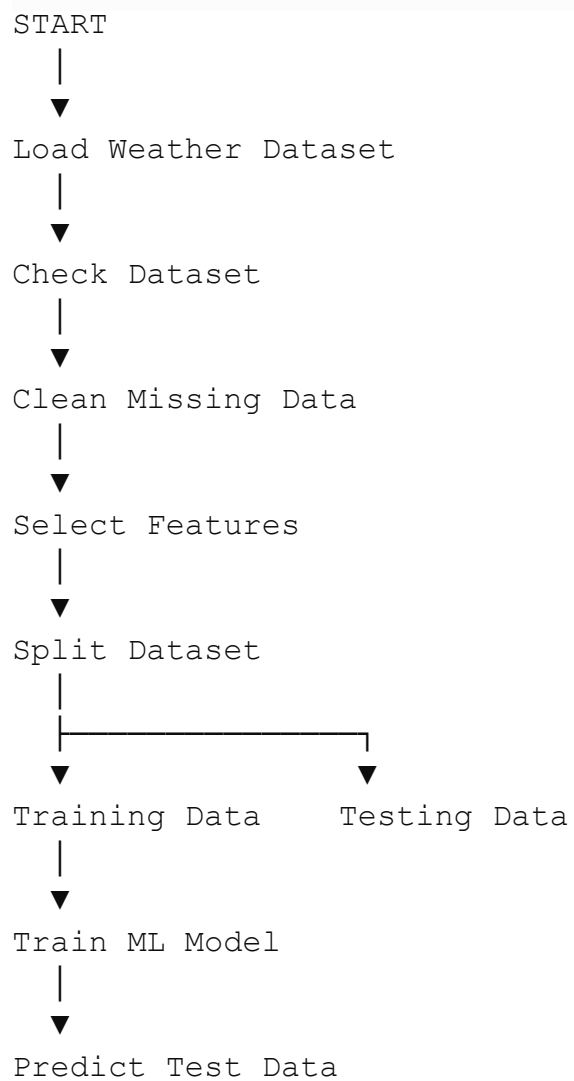
12. System Architecture

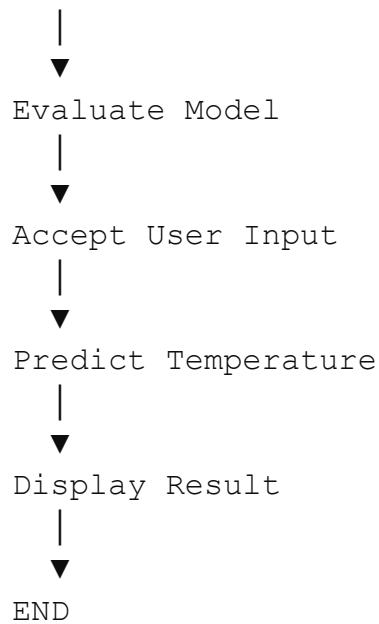
You can put this diagram into your report.





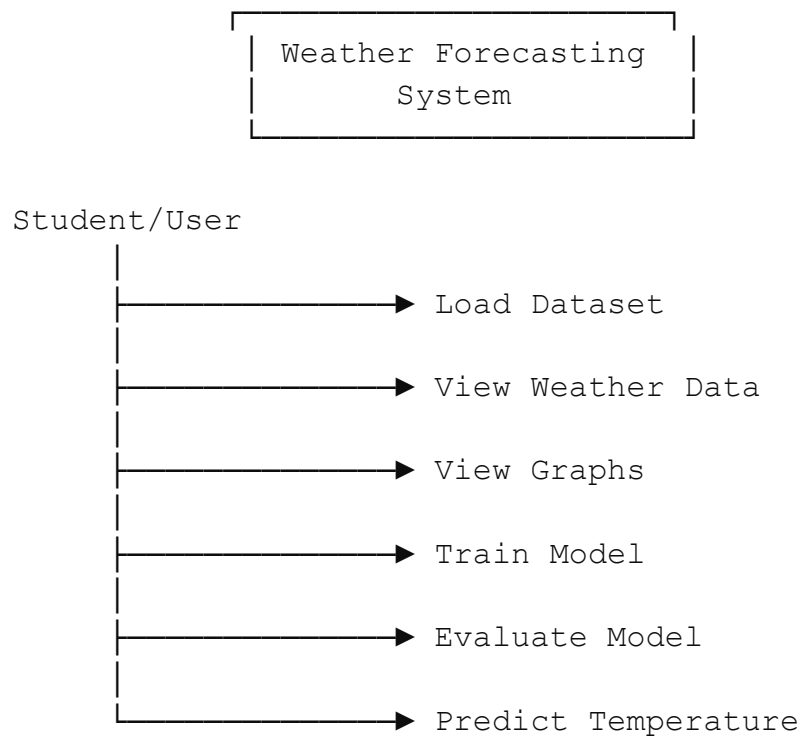
13. Workflow Diagram





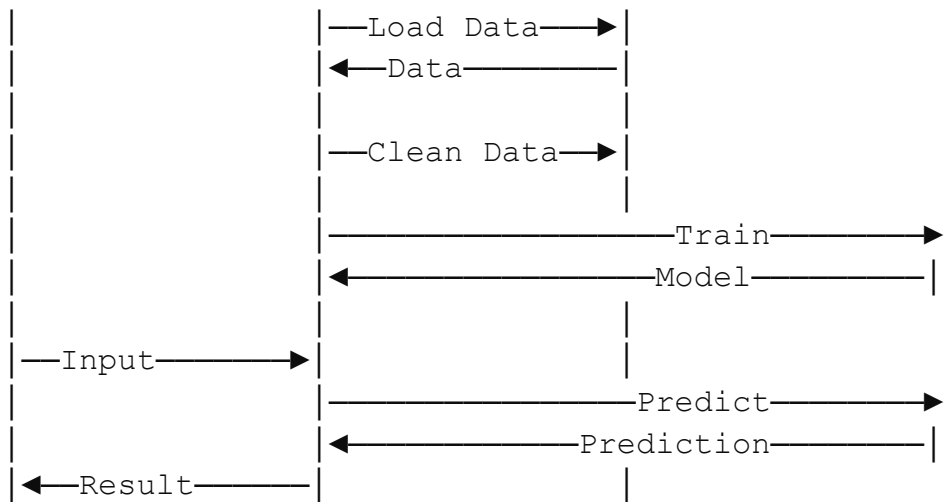
14. Use Case Diagram

You can recreate this using draw.io, Lucidchart, or another diagram tool.



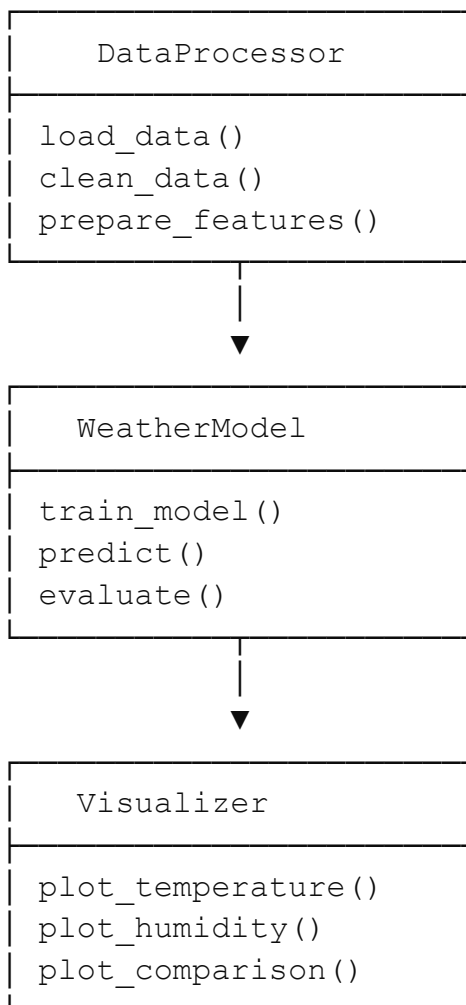
15. Sequence Diagram





16. Class/Component Diagram

Even though this is a beginner project, you can represent the components like this:



17. Project Folder Structure

This is important because your requirement says 5–10 meaningful modules/classes/files.

Use this structure:

```
weather-forecasting-ml/  
├── data/  
│   └── weather.csv  
├── src/  
│   ├── __init__.py  
│   ├── data_processor.py  
│   ├── visualizer.py  
│   ├── model.py  
│   ├── predictor.py  
│   └── main.py  
├── tests/  
│   ├── __init__.py  
│   └── test_model.py  
├── outputs/  
│   ├── temperature_plot.png  
│   ├── humidity_plot.png  
│   └── prediction_results.csv  
├── requirements.txt  
├── README.md  
├── statement.md  
├── .gitignore  
└── LICENSE
```

This gives you a clean and modular project rather than one giant Python file.

18. Python Code

src/data_processor.py

```
import pandas as pd
```

```

class DataProcessor:

    def __init__(self, file_path):
        self.file_path = file_path

    def load_data(self):
        try:
            data = pd.read_csv(self.file_path)
            return data
        except FileNotFoundError:
            print("Error: Weather dataset was not found.")
            return None

    def clean_data(self, data):
        if data is None:
            return None

        required_columns = [
            "Temperature",
            "Humidity",
            "Pressure",
            "WindSpeed",
            "Rainfall"
        ]

        for column in required_columns:
            if column not in data.columns:
                raise ValueError(
                    f"Required column '{column}' is missing."
                )

        data = data.dropna()

        return data

    def prepare_data(self, data):
        features = [
            "Humidity",
            "Pressure",
            "WindSpeed",
            "Rainfall"
        ]

```

```
X = data[features]
y = data["Temperature"]

return X, y
```

19. Machine Learning Model

Create:

src/model.py

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)
import numpy as np

class WeatherModel:

    def __init__(self):
        self.model = LinearRegression()

    def split_data(self, X, y):

        return train_test_split(
            X,
            y,
            test_size=0.2,
            random_state=42
        )

    def train(self, X_train, y_train):

        self.model.fit(X_train, y_train)

    def predict(self, X_test):

        return self.model.predict(X_test)
```

```

def evaluate(self, y_test, predictions):

    mae = mean_absolute_error(
        y_test,
        predictions
    )

    mse = mean_squared_error(
        y_test,
        predictions
    )

    rmse = np.sqrt(mse)

    r2 = r2_score(
        y_test,
        predictions
    )

    return mae, mse, rmse, r2

```

20. Visualization Module

Create:

src/visualizer.py

```

import matplotlib.pyplot as plt

class Visualizer:

    @staticmethod
    def plot_temperature(data):

        plt.figure(figsize=(10, 5))

        plt.plot(
            data["Temperature"],
            color="red"
        )

        plt.title("Temperature Over Time")
        plt.xlabel("Day")

```

```

plt.ylabel("Temperature (°C)")

plt.tight_layout()
plt.savefig("outputs/temperature_plot.png")
plt.show()

@staticmethod
def plot_humidity(data):

    plt.figure(figsize=(10, 5))

    plt.plot(
        data["Humidity"],
        color="blue"
    )

    plt.title("Humidity Over Time")
    plt.xlabel("Day")
    plt.ylabel("Humidity (%)")

    plt.tight_layout()
    plt.savefig("outputs/humidity_plot.png")
    plt.show()

@staticmethod
def plot_actual_vs_predicted(actual, predicted):

    plt.figure(figsize=(8, 5))

    plt.scatter(
        actual,
        predicted,
        color="green"
    )

    plt.xlabel("Actual Temperature")
    plt.ylabel("Predicted Temperature")
    plt.title("Actual vs Predicted Temperature")

    plt.tight_layout()
    plt.savefig(
        "outputs/actual_vs_predicted.png"
    )

```

```
plt.show()
```

21. Prediction Module

Create:

src/predictor.py

```
class Predictor:

    def __init__(self, model):
        self.model = model

    def predict_temperature(
        self,
        humidity,
        pressure,
        wind_speed,
        rainfall
    ):

        input_data = [[
            humidity,
            pressure,
            wind_speed,
            rainfall
        ]]

        prediction = self.model.predict(input_data)

        return prediction[0]
```

22. Main Program

Create:

src/main.py

```
from data_processor import DataProcessor
from model import WeatherModel
from visualizer import Visualizer
from predictor import Predictor
```

```

def main():

    print("=" * 50)
    print("        WEATHER FORECASTING SYSTEM")
    print("=" * 50)

    # Load and process dataset
    processor = DataProcessor(
        "data/weather.csv"
    )

    data = processor.load_data()

    if data is None:
        return

    data = processor.clean_data(data)

    print("\nDataset loaded successfully.")
    print("\nFirst five records:")
    print(data.head())

    # Prepare data
    X, y = processor.prepare_data(data)

    # Create ML model
    weather_model = WeatherModel()

    X_train, X_test, y_train, y_test = (
        weather_model.split_data(X, y)
    )

    # Train model
    weather_model.train(
        X_train,
        y_train
    )

    print("\nModel training completed.")

    # Test prediction
    predictions = weather_model.predict(X_test)

```

```

# Evaluation
mae, mse, rmse, r2 = (
    weather_model.evaluate(
        y_test,
        predictions
    )
)

print("\nMODEL EVALUATION")
print("-" * 30)
print(f"MAE    : {mae:.2f}")
print(f"MSE    : {mse:.2f}")
print(f"RMSE   : {rmse:.2f}")
print(f"R²     : {r2:.2f}")

# Visualization
Visualizer.plot_temperature(data)
Visualizer.plot_humidity(data)

Visualizer.plot_actual_vs_predicted(
    y_test,
    predictions
)

# User prediction
print("\nEnter weather information")
print("-" * 30)

try:

    humidity = float(
        input("Humidity (%): ")
    )

    pressure = float(
        input("Pressure (hPa): ")
    )

    wind_speed = float(
        input("Wind Speed (km/h): ")
    )

    rainfall = float(

```

```

        input("Rainfall (mm): ")
    )

    predictor = Predictor(
        weather_model.model
    )

    result = predictor.predict_temperature(
        humidity,
        pressure,
        wind_speed,
        rainfall
    )

    print("\n" + "=" * 40)
    print(
        f"Predicted Temperature: "
        f"{result:.2f} °C"
    )
    print("=" * 40)

except ValueError:

    print(
        "Invalid input. "
        "Please enter numerical values."
    )

if __name__ == "__main__":
    main()

```

23. Requirements File

Create:

requirements.txt

```

pandas
numpy
matplotlib
scikit-learn

```

You can install everything with:

```
pip install -r requirements.txt
```

24. Testing

Create:

tests/test_model.py

```
import pandas as pd
from sklearn.linear_model import LinearRegression

def test_model_training():

    X = pd.DataFrame({
        "Humidity": [50, 60, 70, 80],
        "Pressure": [1010, 1011, 1012, 1013],
        "WindSpeed": [5, 6, 7, 8],
        "Rainfall": [0, 1, 2, 3]
    })

    y = pd.Series([
        25,
        26,
        27,
        28
    ])

    model = LinearRegression()

    model.fit(X, y)

    prediction = model.predict(
        [[65, 1011, 6.5, 1]]
    )

    assert len(prediction) == 1
    assert prediction[0] > 0
```

Run:

```
pytest
```

If you want to use pytest, add it to requirements.txt:

```
pytest
```

25. Example Dataset

Your data/weather.csv can initially look like:

```
Date, Temperature, Humidity, Pressure, WindSpeed, Rainfall
2025-01-01, 22.5, 65, 1012, 6.2, 0
2025-01-02, 23.1, 63, 1011, 7.1, 0
2025-01-03, 24.0, 60, 1010, 8.2, 0
2025-01-04, 25.2, 58, 1009, 7.5, 1
2025-01-05, 24.7, 62, 1011, 6.8, 2
2025-01-06, 23.8, 66, 1013, 5.5, 4
2025-01-07, 22.9, 70, 1014, 5.1, 5
2025-01-08, 24.3, 64, 1012, 6.7, 1
2025-01-09, 26.1, 55, 1008, 9.0, 0
2025-01-10, 27.0, 52, 1007, 9.5, 0
```

Scope

The project covers:

- Weather data collection
- Data preprocessing
- Data visualization
- Machine learning
- Temperature prediction
- Model evaluation

The project is intended for educational purposes and is not a replacement for professional weather forecasting systems.

Overview

Weather Forecasting Using Machine Learning is a beginner-friendly Python project that uses historical weather data and Linear Regression to predict temperature.

Features

- Load weather data from CSV
- Clean and preprocess data
- Analyze weather information
- Generate graphs
- Train Linear Regression model
- Evaluate model
- Predict temperature from user input
- Handle invalid input

Technologies

- Python
- Pandas
- NumPy
- Matplotlib
- Scikit-learn
- Git
- GitHub

Project Structure

weather-forecasting-ml/

```
|— data/  
|— src/  
|— tests/  
|— outputs/  
|— requirements.txt  
|— README.md  
|— statement.md
```

Run:

```
python src/main.py
```

Testing

Run:

pytest

Machine Learning Model

The project uses Linear Regression.

The input features are:

- Humidity
- Pressure
- Wind Speed
- Rainfall

The target variable is:

- Temperature

Evaluation Metrics

The model is evaluated using:

- MAE
- MSE
- RMSE
- R^2 Score

Disclaimer

This project is developed for educational purposes.
It should not be considered a professional weather forecasting system.

30. Introduction

You can write:

Weather forecasting is an important application of data analysis and computational techniques. Weather conditions such as temperature, humidity, pressure, wind speed

and rainfall change over time. Machine learning provides a method for learning patterns from historical data and using those patterns to make predictions.

This project develops a beginner-friendly weather forecasting system using Python and machine learning. Historical weather observations are processed and used to train a Linear Regression model. The trained model is then used to predict temperature based on weather-related input features.

Why Python?

Python has a simple syntax and provides widely used libraries for data processing, visualization, and machine learning.

Why Pandas?

Pandas provides convenient data structures and functions for reading and processing CSV data.

Why Matplotlib?

Matplotlib allows the project to visually represent weather trends.

Why Linear Regression?

Linear Regression is simple to understand and suitable for introducing regression-based machine learning to beginners.

Why CSV?

CSV is lightweight, easy to understand, and sufficient for a small educational learned how to clean data, select features, split data into training and testing sets, train a machine-learning model, and evaluate its performance. I also learned the importance of modular programming, testing, documentation, and version control using Git and GitHub dataset and claim that it represents real weather forecasting. Use a legitimate historical weather dataset with enough observations to train and, React, Docker, etc. A clean Python project with a real dataset, Linear Regression, visualizations, testing, dataset.

Why modular code?

Separating data processing, visualization, modeling, prediction, and testing makes the project easier to understand and maintain.

CSV Dataset
↓
Pandas
↓
Data Cleaning
↓
Feature Selection
↓
Train/Test Split
↓
Linear Regression
↓
Prediction
↓
Evaluation
↓
Visualization

Testing Approach

Create a table like this:

Test Case		Input	Expected Result
TC01	Valid CSV		Dataset loads
TC02	Missing CSV		Error message
TC03	Valid weather values		Temperature prediction
TC04	Text instead of number		Input error

TC05	Missing required column	Validation error
TC06	Model training	Model successfully trained

Challenges Faced

Data Cleaning

Historical datasets may contain missing or inconsistent values.

Understanding Machine Learning

Understanding the difference between training data and testing data can initially be difficult.

Choosing Features

Not every available weather column is necessarily required for the first version.

Model Evaluation

Understanding MAE, RMSE, and R^2 requires learning how prediction errors are measured.

Python Libraries

Learning Pandas and Scikit-learn APIs can be challenging initially.

Learnings and Key Takeaways

During the development of this project, I learned how to work with datasets using Python and Pandas. I learned how to clean data, select features, split data into training and testing sets, train a machine-learning model, and evaluate its performance. I also learned the importance of modular programming, testing, documentation, and version control using Git and GitHub.

Future Enhancements

This is a good section for my presentation because it shows how the project could be developed further.

Possible enhancements:

- Use real-time weather APIs.
- Add rainfall prediction.
- Add humidity prediction.
- Compare multiple ML algorithms.
- Add a graphical user interface.
- Develop a web application.
- Add database support.
- Use larger datasets.
- Add automatic daily forecasting.
- Experiment with advanced time-series models.
- Deploy the application online.

References

For example:

- [Python documentation](#)
- [Pandas documentation](#)
- [NumPy documentation](#)
- [Matplotlib documentation](#)
- [Scikit-learn documentation](#)